

Angular Velocity Calibration

Angular Velocity Calibration

1. Course Content
2. Preparations
 - 2.1 Content Description
 - 2.2 Start the Agent
3. Run the Example
 - 3.1 Start the Program
 - 3.2 Start Calibration
 - 3.3 Write Calibration Parameters to the Chassis
4. Source Code Analysis
 - 4.1 View Node Relationship Graph
 - 4.2 Source Code Analysis

1. Course Content

1. Learn how to calibrate the robot's angular velocity. After running the angular velocity calibration program and clicking "start" in the visualization interface, the robot chassis will begin to rotate and will stop when the error is less than the tolerance value.

[!IMPORTANT]

- The vehicle has been calibrated before leaving the factory. Under normal circumstances, calibration is not necessary. If there is a significant error due to special circumstances, you can perform angular velocity calibration.

2. Preparations

2.1 Content Description

This lesson uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host machine, enter the command to get into the Docker container, and then input the commands mentioned in this lesson in the terminal within the Docker container. A tutorial on how to enter the Docker container from the host can be found in this product's guide [00.Configuration and Operation Guide]-[3.Enter the Docker (For RPI5)]. For ORIN and RDK boards, you can directly open a terminal and enter the commands mentioned in this lesson.

2.2 Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
subscriber created | client_key: 0x1D229138, subscriber_id: 0x000(4), participant_id: 0x000(1)
[1764917701.432450] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x000(6), subscriber_id: 0x000(4)
[1764917701.436381] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x1D229138, topic_id: 0x006(2), participant_id: 0x000(1)
[1764917701.439564] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), participant_id: 0x000(1)
[1764917701.444080] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscriber_id: 0x001(4)
[1764917701.449158] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_id: 0x000(1)
[1764917701.452629] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), participant_id: 0x000(1)
[1764917701.457477] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscriber_id: 0x002(4)
```

3. Run the Example

3.1 Start the Program

Open a terminal on the vehicle and run the angular velocity calibration node:

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Enter the command,

```
ros2 launch m3_bringup calibration.launch.py calibration_mode:=angular
```

[!TIP]

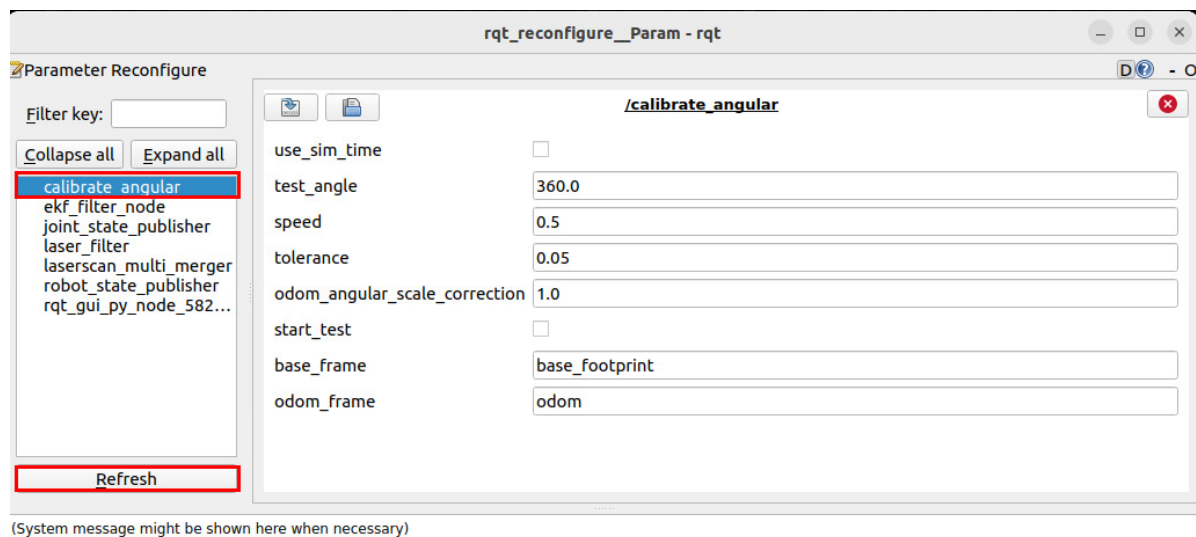
`calibration_mode` is a launch parameter with two options:

- `linear`: The default parameter, which starts linear velocity calibration.
- `angular`: Starts angular velocity calibration.

Then, open another terminal and start the dynamic parameter reconfigure tool:

```
ros2 run rqt_reconfigure rqt_reconfigure
```

Click the **calibrate_angular** node in the node options on the left:



[!WARNING]

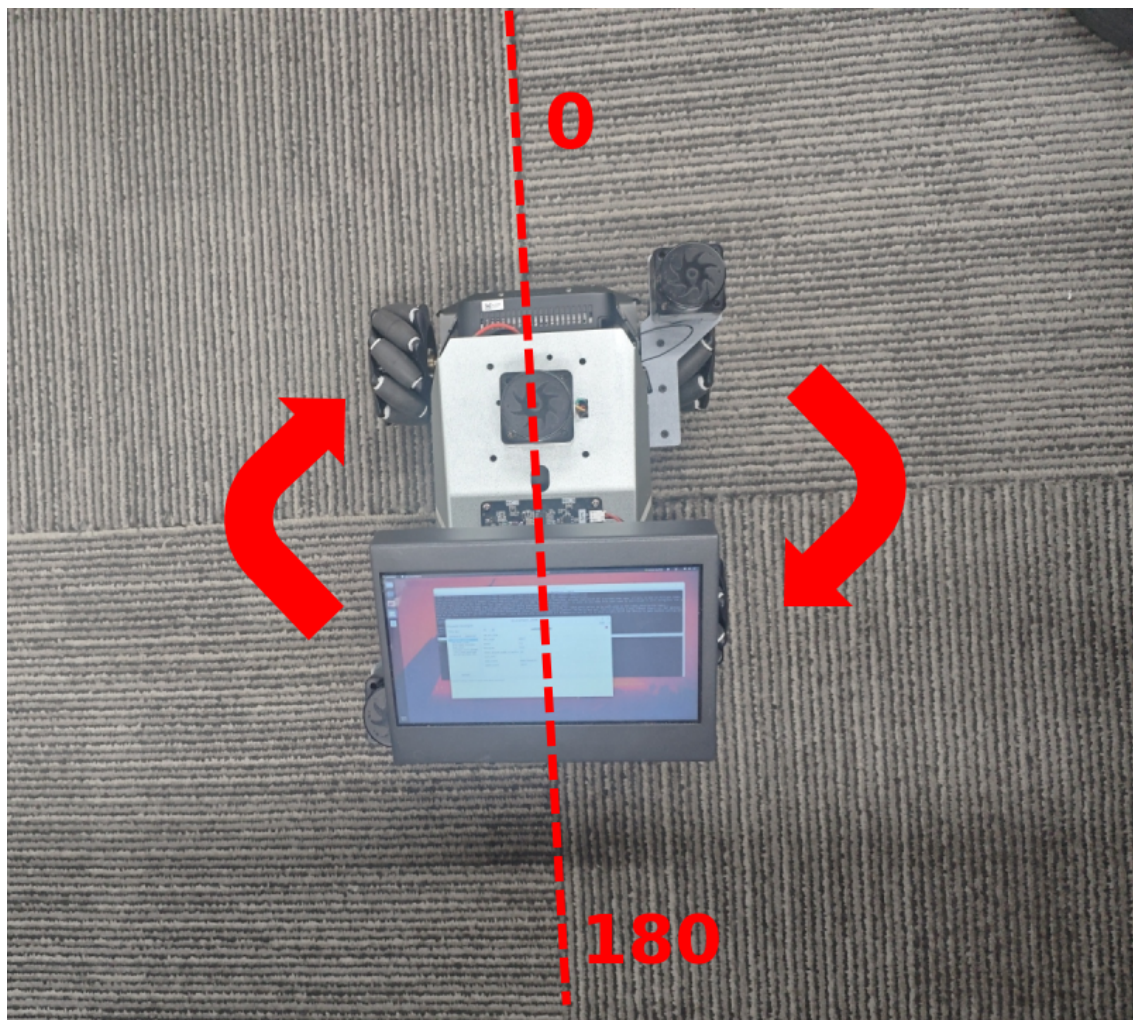
- When you first open it, the above node may not be present. Click **Refresh** to see all the nodes. The displayed **calibrate_angular** node is the one for calibrating angular velocity.

RQT interface parameter description:

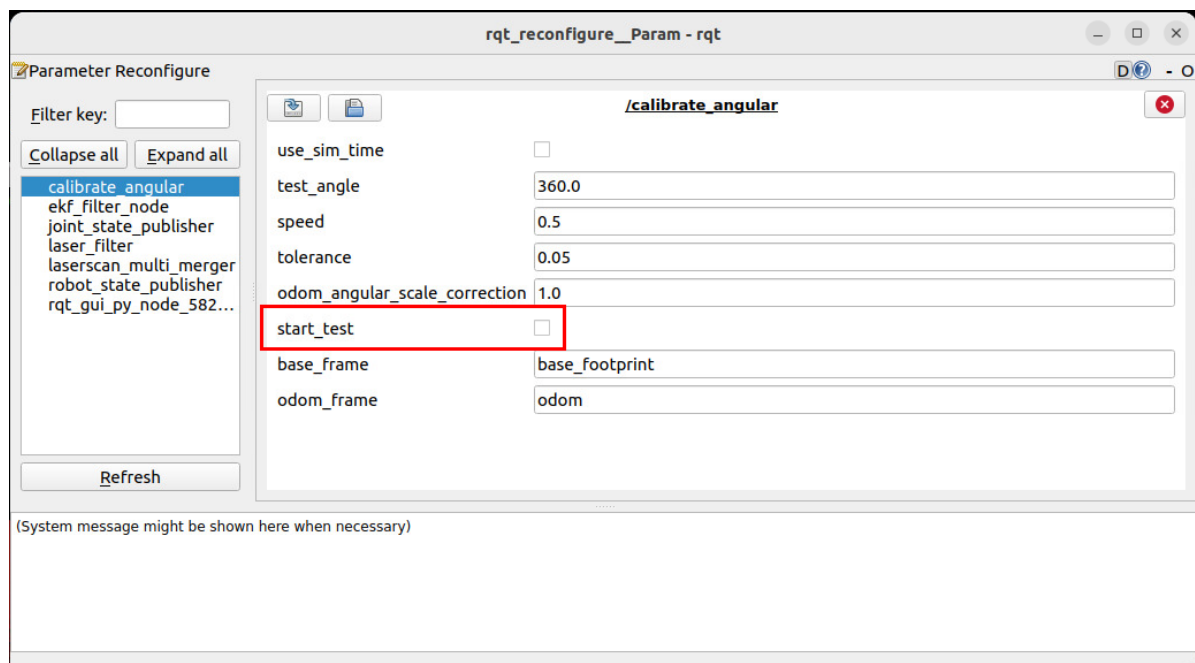
- **test_angle**: The angle for the calibration test. Here, it's set to rotate 360 degrees.
- **speed**: The angular velocity.
- **tolerance**: The allowable error tolerance.
- **odom_angular_scale_correction**: The angular velocity scale factor. If the test results are not ideal, modify this value.
- **start_test**: The test switch.
- **base_frame**: The name of the base coordinate frame.
- **odom_frame**: The name of the odometry coordinate frame.

3.2 Start Calibration

1. In the **rqt_reconfigure** interface, select the **calibrate_angular** node. For precise calibration, you can use a protractor for measurement. For a rough calibration, you can use a line on the floor as a reference.



2. In `rqt_reconfigure`, check the box next to **start_test** to begin the calibration.



3. The car will listen to the TF transform between `base_footprint` and `odom`, calculate the theoretical angle of rotation, and when the error is less than `tolerance`, it will publish a stop command.


```

jetson@yahboom: ~
jetson@yahboom: ~ 168x16
[calibrate_angular-6] [INFO] [1764918810.991998452] [calibrate_angular]: turn_angle: 5.6649667989405295, error: 0.6182185082390568
[calibrate_angular-6] [INFO] [1764918811.091824584] [calibrate_angular]: turn_angle: 5.733839780636555, error: 0.5493455265430311
[calibrate_angular-6] [INFO] [1764918811.192340919] [calibrate_angular]: turn_angle: 5.770889840098752, error: 0.5122954670808344
[calibrate_angular-6] [INFO] [1764918811.293686504] [calibrate_angular]: turn_angle: 5.831127565603346, error: 0.45205774157624035
[calibrate_angular-6] [INFO] [1764918811.393948267] [calibrate_angular]: turn_angle: 5.871226479297958, error: 0.4119588278816284
[calibrate_angular-6] [INFO] [1764918811.494211631] [calibrate_angular]: turn_angle: 5.929935031260065, error: 0.3532502759195211
[calibrate_angular-6] [INFO] [1764918811.592824975] [calibrate_angular]: turn_angle: 5.964533770911194, error: 0.3186515362683924
[calibrate_angular-6] [INFO] [1764918811.693254298] [calibrate_angular]: turn_angle: 6.018105512875449, error: 0.2650797943041372
[calibrate_angular-6] [INFO] [1764918811.795694072] [calibrate_angular]: turn_angle: 6.067053837733589, error: 0.21613146944599748
[calibrate_angular-6] [INFO] [1764918811.892656435] [calibrate_angular]: turn_angle: 6.121867789815846, error: 0.16131751736374067
[calibrate_angular-6] [INFO] [1764918811.993319495] [calibrate_angular]: turn_angle: 6.154505228672754, error: 0.12868007850683227
[calibrate_angular-6] [INFO] [1764918812.093870021] [calibrate_angular]: turn_angle: 6.203664715634741, error: 0.07952059154484559
[calibrate_angular-6] [INFO] [1764918812.192287495] [calibrate_angular]: turn_angle: 6.264488977816323, error: 0.018696329363263153
[calibrate_angular-6] [INFO] [1764918812.193785957] [calibrate_angular]: turn_angle: 6.264488977816323, error: 0.018696329363263153
[calibrate_angular-6] [INFO] [1764918812.196836739] [calibrate_angular]: done

```

4. If the actual angle the car rotates is not 360 degrees, modify the `odom_angular_scale_correction` parameter in RQT. After modifying it, click on a blank area, then uncheck and recheck `start_test` to reset it, and then click `start_test` again to calibrate.
5. Repeat the above process to find a suitable `odom_angular_scale_correction` value.

3.3 Write Calibration Parameters to the Chassis

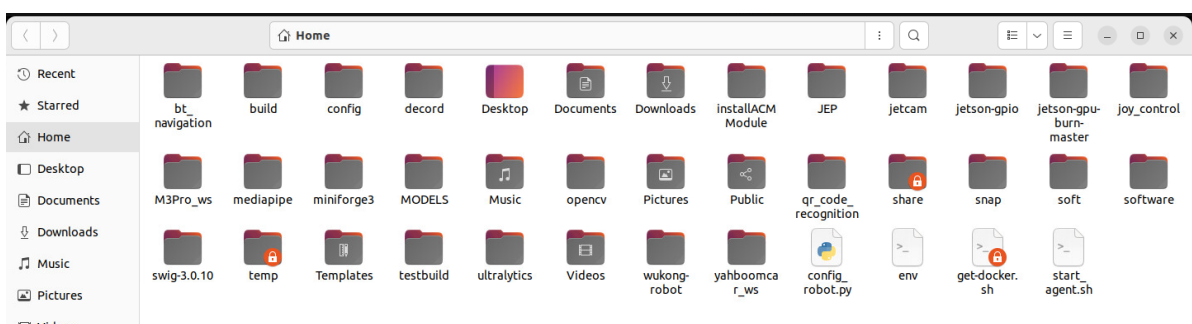
1. To write parameters to the chassis, you must first disconnect the chassis agent by pressing **Ctrl+C** or closing the chassis connection agent terminal.

```

Terminal
d: 0x000(1)
[6222.246556] info | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x002(6), subscriber_id
: 0x002(4)
[6222.252366] info | ProxyClient.cpp | create_topic | topic c
reated | client_key: 0x77BF8684, topic_id: 0x008(2), participant_id: 0x
000(1)
[6222.255974] info | ProxyClient.cpp | create_subscriber | subscri
ber created | client_key: 0x77BF8684, subscriber_id: 0x003(4), participant_i
d: 0x000(1)
[6222.261071] info | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x003(6), subscriber_id
: 0x003(4)
[6222.265578] info | ProxyClient.cpp | create_topic | topic c
reated | client_key: 0x77BF8684, topic_id: 0x009(2), participant_id: 0x
000(1)
[6222.268942] info | ProxyClient.cpp | create_subscriber | subscri
ber created | client_key: 0x77BF8684, subscriber_id: 0x004(4), participant_i
d: 0x000(1)
[6222.275467] info | ProxyClient.cpp | create_datareader | datarea
der created | client_key: 0x77BF8684, datareader_id: 0x004(6), subscriber_id
: 0x004(4)
^C[ros2run]: Interrupt

```

2. Open the `config_robot.py` file in the home directory on the vehicle.



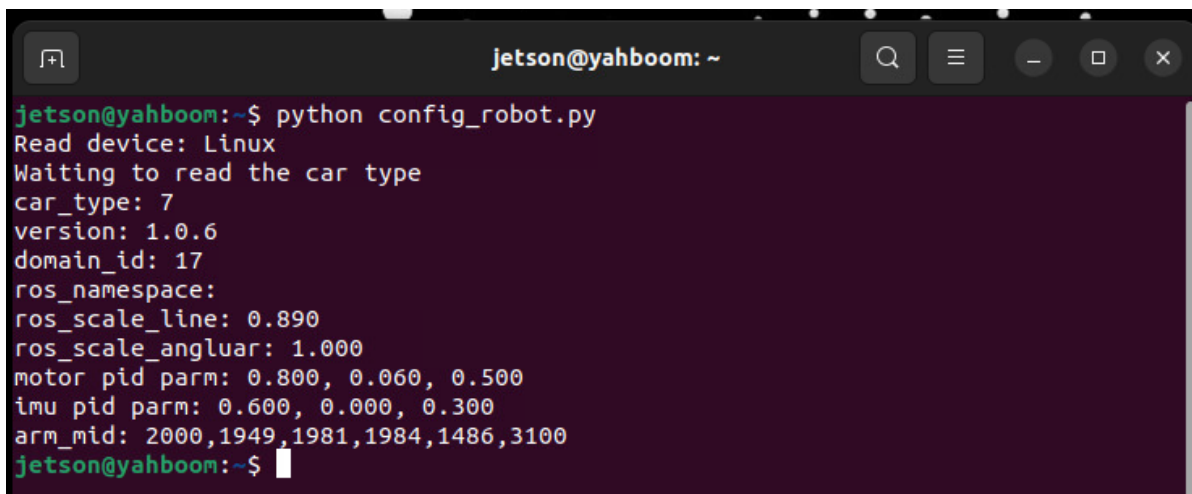
3. Uncomment line 552, fill in the previous calibration coefficient in the parentheses of **robot.set_ros_scale_angluar(xx)**, and then click **save**.



```
538     car_type = robot.read_car_type()
539     if car_type is not None:
540         print("car_type:", car_type)
541         if int(car_type) == robot.CAR_TYPE:
542             break
543     time.sleep(.5)
544
545
546     # robot.set_ros_domain_id(30)
547     # robot.set_ros_namespace("")
548     # robot.set_motor_pid_parm(0.8, 0.06, 0.5)
549     # robot.set_imu_yaw_pid_parm(0.6, 0, 0.3)
550
551     robot.set_ros_scale_line(0.89)
552     robot.set_ros_scale_angluar(1.0)
```

4. Open a terminal on the vehicle and enter the command:

```
python3 config_robot.py
```



```
jetson@yahboom:~$ python3 config_robot.py
Read device: Linux
Waiting to read the car type
car_type: 7
version: 1.0.6
domain_id: 17
ros_namespace:
ros_scale_line: 0.890
ros_scale_angluar: 1.000
motor pid parm: 0.800, 0.060, 0.500
imu pid parm: 0.600, 0.000, 0.300
arm_mid: 2000,1949,1981,1984,1486,3100
jetson@yahboom:~$
```

Wait for the parameters to be written. The `ros_scale_angluar:1.000` in the terminal's printed information is the written parameter. The chassis angular velocity calibration is complete.

4. Source Code Analysis

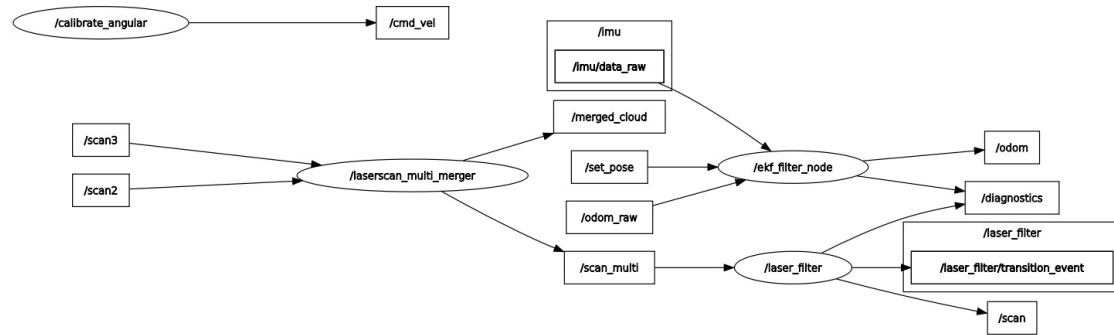
Source code path:

```
~/yahboomM3_ws/code_ws/src/m3-common-demo/m3-common-demo/calibration/calibrate_angular.py
```

4.1 View Node Relationship Graph

Open a terminal on the virtual machine and enter the command:

```
ros2 run rqt_graph rqt_graph
```



In the node relationship diagram above:

- The **/ekf_filter_node** subscribes to the chassis's raw odometry **/odom_raw** and IMU data **/imu/data_raw**, and after data fusion, it publishes the **/odom** topic.
- The **calibrate_angular** node listens to the TF transform from **odom** to **base_footprint** and publishes the **/cmd_vel** topic to control the robot chassis movement.

4.2 Source Code Analysis

Listening to the TF coordinate transformation is implemented in the `get_odom_angle` method of the `CalibrateAngular` class:

```
def get_odom_angle(self):
    try:
        now = rclpy.time.Time()
        rot = self.tf_buffer.lookup_transform(
            self.base_frame,
            self.odom_frame,
            now,
            timeout=rclpy.duration.Duration(seconds=1.0))
        cac1_rot = PyKDL.Rotation.Quaternion(rot.transform.rotation.x,
            rot.transform.rotation.y, rot.transform.rotation.z,
            rot.transform.rotation.w)
        #print("cac1_rot: ",cac1_rot)
        angle_rot = cac1_rot.GetRPY()[2]
        #print("angle_rot: ",angle_rot)

    except (LookupException, ConnectivityException, ExtrapolationException):
        # self.get_logger().info('transform not ready')
        return
```

Judging the robot chassis's rotation angle and controlling its movement is implemented in the `on_timer` (timer callback function) method of the `CalibrateAngular` class:

```
def on_timer(self):
    self.start_test =
    self.get_parameter('start_test').get_parameter_value().bool_value
    self.odom_angular_scale_correction =
    self.get_parameter('odom_angular_scale_correction').get_parameter_value().double_value
```

```

        self.test_angle =
self.get_parameter('test_angle').get_parameter_value().double_value
        self.test_angle = radians(self.test_angle) # Convert degrees to radians
        self.speed = self.get_parameter('speed').get_parameter_value().double_value
        move_cmd = Twist()
        self.test_angle *= self.reverse
        #self.test_angle *= self.reverse
        #self.error = self.test_angle - self.turn_angle
        if self.start_test:
            self.error = self.turn_angle - self.test_angle
            if abs(self.error) > self.tolerance :
                #move_cmd.linear.x = 0.2
                move_cmd.angular.z = copysign(self.speed, self.error)
                #print("angular: ",move_cmd.angular.z)
                self.cmd_vel.publish(move_cmd)
                self.odom_angle = self.get_odom_angle()
                self.delta_angle = self.odom_angular_scale_correction *
self.normalize_angle(self.odom_angle - self.first_angle)
                #print("delta_angle: ",self.delta_angle)
                self.turn_angle += self.delta_angle
                print("turn_angle: ",self.turn_angle,flush=True)
                #self.error = self.test_angle - self.turn_angle
                print("error: ",self.error,flush=True)
                self.first_angle = self.odom_angle

                #print("first_angle: ",self.first_angle)
            else:
                self.error = 0.0
                self.turn_angle = 0.0
                print("done",flush=True)
                self.first_angle = 0
                self.reverse = -self.reverse
                self.start_test =
rclpy.parameter.Parameter('start_test',rclpy.Parameter.Type.BOOL,False)
                all_new_parameters = [self.start_test]
                self.set_parameters(all_new_parameters)
        else:
            self.error = 0.0
            self.cmd_vel.publish(Twist())
            self.turn_angle = 0.0
            self.start_test =
rclpy.parameter.Parameter('start_test',rclpy.Parameter.Type.BOOL,False)

            all_new_parameters = [self.start_test]
            self.set_parameters(all_new_parameters)

```